

CPP NOTES – DAY 35

Copy in C++?

In C++, when you assign or pass an object, a **copy of the object** is made using:

- **Copy constructor**
- Or **assignment operator**

Now, how the copy behaves depends on whether it is a **shallow** or a **deep** copy.

1. What is a Shallow Copy?

A **shallow copy** copies **only the pointer values**, not the actual data they point to. That means **both the original and copied object share the same memory**.

◆ Problems:

- Modifying one object affects the other.
- When destructors run, **double deletion** may happen.

```

class Shallow {
public:
    int* data;

    Shallow(int value) {
        data = new int(value);
    }

    ~Shallow() {
        delete data;
    }
};

int main() {
    Shallow obj1(10);
    Shallow obj2 = obj1; // Shallow copy

    *obj2.data = 20;

    cout << *obj1.data << endl; // Outputs 20 (oops!)

    return 0;
}

```

2. What is a Deep Copy?

A **deep copy** creates a new copy of the data pointed to, not just the pointer. Each object has its **own separate memory**.

✅ Advantages:

- Objects don't interfere with each other.
- No shared memory, so no double deletion.

```

class Deep {
public:
    int* data;

    Deep(int value) {
        data = new int(value);
    }

    // Deep Copy Constructor
    Deep(const Deep& other) {
        data = new int(*other.data); // Create new memory and copy value
    }

    ~Deep() {
        delete data;
    }
};

int main() {
    Deep obj1(10);
    Deep obj2 = obj1; // Deep copy

    *obj2.data = 20;

    cout << *obj1.data << endl; // Outputs 10
    cout << *obj2.data << endl; // Outputs 20

    return 0;
}

```

Summary Table

Feature	Shallow Copy	Deep Copy
Memory	Shares memory	Allocates new memory
Safety	Risk of double delete	Safe independent objects
Copying	Copies pointer only	Copies actual data
Destructor	May delete same memory twice	Deletes own memory only
Use Case	When object doesn't own pointer	When object owns dynamic memory

When is this Important?

Especially important when:

- Your class uses `new` to allocate memory (dynamic arrays, strings, etc.).
- You need to implement **custom copy constructor and assignment operator**.

FYI:

- *Objects that acts like functions are called Functors.*
- *Pointer that points towards a function is function pointer.*